SSL Project Report

Kurivella Harshith
and
Nemmadi Aakash

29 april 2026

Contents

1 External Libraries, python modules and their usage in project **3**
1.1 List of libraries . . . 3
1.2 List of python modules . . . 3
1.3 Usage of Libraries . . . 3
1.4 Usage of built-in python modules . . . 4

2 Features of the game hub **4**
2.1 Menu . . . 4
2.2 Games in Mini Game Hub . . . 4
2.3 TicTacToe Game . . . 5
2.4 Othello(reversi) Game . . . 6
2.5 ConnectFour Game . . . 7
2.6 Game over menu . . . 7
2.7 Sort By Menu . . . 8
2.8 Statistics . . . 9

3 Code Logic and implementation **9**
3.1 TicTacToe . . . 9
3.1.1 Overview . . . 9
3.1.2 Board Representation . . . 10
3.1.3 Rendering the Board . . . 10

3.1.4 Move Handling 11
3.1.5 Win Detection Logic 11
3.1.6 Game Loop 12
3.1.7 Result Storage 12
3.1.8 Conclusion 12
3.2 Othello 13
3.2.1 Overview 13
3.2.2 Board Representation 13
3.2.3 Initial Setup 13
3.2.4 Direction-Based Move Logic 13
3.2.5 Valid Move and Flip Detection 13
3.2.6 Valid Moves Generation 14
3.2.7 Move Execution 14
3.2.8 Win Condition Logic 14
3.2.9 User Interaction Handling 14
3.2.10 Game Loop 15
3.2.11 Result Storage 15
3.2.12 Conclusion 15
3.3 Connect4 15
3.3.1 Overview 15
3.3.2 Board Representation 15
3.3.3 Rendering the Board 16
3.3.4 Column Selection Logic 16
3.3.5 Move Execution 16
3.3.6 Win Detection Logic 17
3.3.7 Game Loop 17
3.3.8 Result Handling 17
3.3.9 Conclusion 17

4 Hurdles, issues faced during the project and overcoming them 18
4.1 Hurdles 18
4.2 Overcoming the hurdles 18

5 Possible project improvements if extra time is permitted/alloted 19

6 References 19

1 External Libraries, python modules and their usage in project

1.1 List of libraries

- **numpy**
- **pygame**
- **matplotlib**

1.2 List of python modules

- **sys**
- **time**
- **subprocess**

1.3 Usage of Libraries

numpy : -

numpy is main core part of the project. Numpy array is used as board for all the games. Its helper functions for manipulation of its arrays helps to implement the game logic easily other wise would have been difficult to do. Numpy arrays slicing is allows to do operations on the game board, which would be hard when done with nested loops. [6],[5],[2]

pygame : -

In this project, pygame is used to visualize game from normal matrix into design game(board etc), colour the board to make the game more fun; colourful,capturing the click/ inputs from user, change upon making moves, display the winner of the game and allow user to select the metric for sorting the players, various menus etc.[3]

matplotlib : -

matplotlib in this project is used to show the statistics of players in the game hub. It is used to plot the bar graph for the top 5 players based

on the metric selected by user. It is also used to plot the pie chart for the distribution of wins among the players.[1]

1.4 Usage of built-in python modules

sys : -

In this project sys is used to manipulate the arguments passed to the python script i.e, game.py as command-line arguments so that the names of player can be used directly.

time : -

The time module is used in this project to show time,date of game played by two players.

subprocess : -

In this project, it is used to run leaderboard.sh while running main.sh

2 Features of the game hub

2.1 Menu

Once users runs the bash command, A menu named "AAHa Game Hub" appears on interface. The menu displays the all the 3 games available in the mini game hub. When the user selects a game the corresponding game runs.

2.2 Games in Mini Game Hub

The Mini game hub consists of three games: -

1. TicTacToe
2. Othello
3. ConnectFour

Figure 1: AAHa Game Hub menu

2.3 TicTacToe Game

After the user select *"TicTacToe"* game option, the game hub will lead to TicTacToe game interface. The board is of Bluish Grey colour. The game board is a grid of 10,10. Upon clicking the square box in the grid, neon blue cross appears for player 1, for the other player it is neon yellow colour circle. Above the game board, the interface will display symbol (cross for player 1 and circle for player 2) player whose turn is now. An occupied square will not change by either players once the square is clicked by either player. A player wins if consecutive 5 square boxes (either horizontally, vertically or diagonally) have same symbol (neon blue cross for player 1 and neon yellow circle for player 2).There is quit button/option on top the game board. There

is a menu option(in red colour) on top of the game board , if user selects the menu option, the game terminates and it goes back the main game hub menu(AAHa Game Hub).

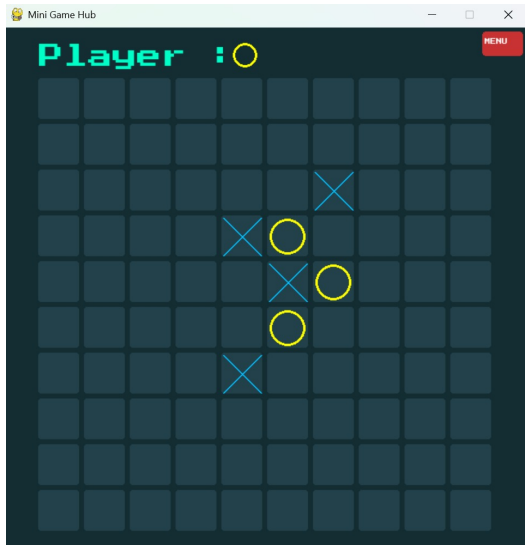

Figure 2: TicTacToe game

2.4 Othello(reversi) Game

After the user select *"Othello"* game option, the game hub will lead to Othello game interface. The board is Green. The game board is a grid of 64 discs (8,8)-standard Othello.Empty discs are shown in dark green circles. The game board will initially have two black circles and two white circles placed at centre of the game board with two same coloured squares arranged diagonally. Above the game board, the interface will show player name, current players dis'c and number of tiles each player's disc occupied.The interface also highlights by adding neon-blue circle to circumference of the discs that are valid for current player. A player will win if he/she has *more* tiles than opponent player when either player doesn't have valid moves left even in case where the game board has empty tiles in it. There is quit button/option on top the game board. There is a menu option(in red colour) on top of the game board , if user selects the menu option, the game terminates and it goes back the main game hub menu(AAHa Game Hub).

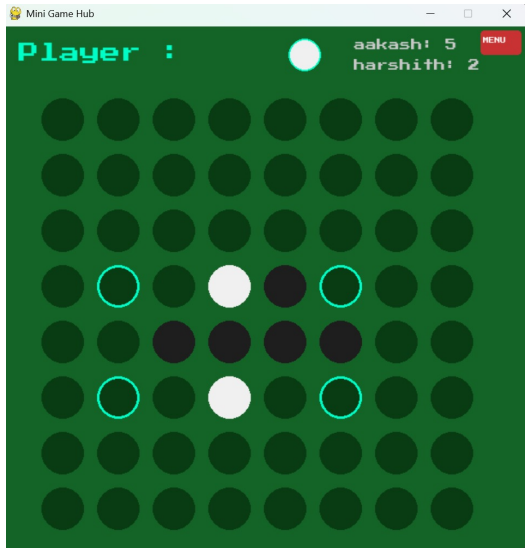

Figure 3: Othello(reversi) game

2.5 ConnectFour Game

Once User select/clicks *"Connect4"* game option, the game hub will lead to Connectfour game interface. The game board is maroon colour consisting an array of dark maroon discs(7*7) The game board is a grid of 49 discs(7,7). Once player clicks a incomplete column, the red disc occupies the lowest unoccupied position. For player 2 it is yellow colour disc. A player wins once the 4 consecutive discs(horizontally, vertically or diagonally) . There is quit button/option on top the game board. There is a menu option(in red colour) on top of the game board , if user selects the menu option, the game terminates and it goes back the main game hub menu (AAHa Game Hub).

2.6 Game over menu

After a player wins/draws the game, a menu named "Game Over" appears. It displays the winner of the game(shows draw in case of draw), shows option

1. Leaderboard
2. Statistics
3. Restart

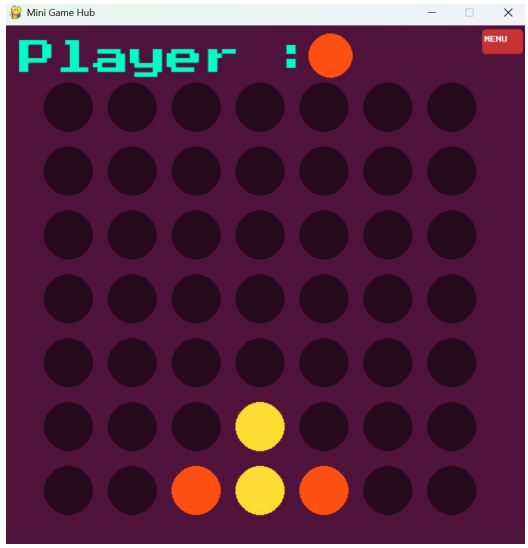

Figure 4: Connect4 game

2.7 Sort By Menu

If user selects Leaderboard option, a menu appears named "Sort By" asking the metric to display all the players based on metric the user chose to sort the leaderboard. Sorting metrics include-

1. User/player's name
2. Total Number of games a player played
3. Number of games won by players
4. Number of games lost by players
5. Ratio of number of games won to the number of games lost

Once User selects the sorting metric/criteria, A Leaderboard is printed to terminal, showing all players sorted based on selected criteria/metric. The leaderboard will display Player names, their number of wins, number of losses, number of games played and ratio of number of wins and losses.

Figure 5: Game over menu

2.8 Statistics

If user selects Statistics option, the game hub will show a pi chart showing the number of times each game is played, two bar graphs showing - one bar graph shows the top 5 best players whereas the other shows top 5 worst players. Below the graphs there is menu showing an option: Back, Menu. If user clicks/selects back option, it will come back to Game over menu.

3 Code Logic and implementation

3.1 TicTacToe

3.1.1 Overview

The Tic-Tac-Toe game is implemented using a 10×10 grid where two players take turns placing their symbols. Player 1 is represented by 'X' and Player 2 by 'O'. The objective is to align five consecutive symbols horizontally, vertically, or diagonally.

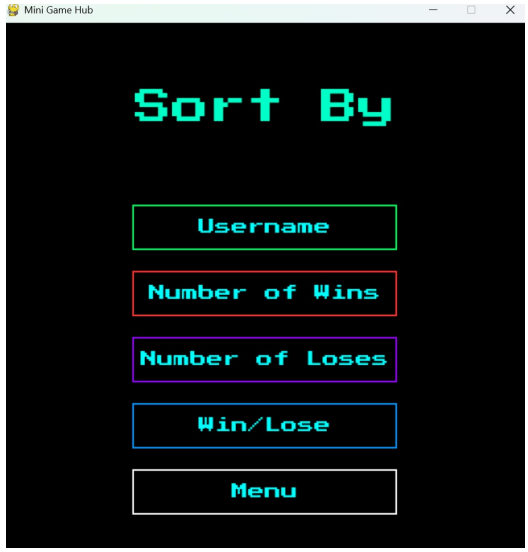

Figure 6: Sort By menu

3.1.2 Board Representation

The game board is represented using a 2D NumPy array:

- Value 0 represents an empty cell
- Value 1 represents Player 1 (X)
- Value 2 represents Player 2 (O)

3.1.3 Rendering the Board

The `draw_board()` function is responsible for:

- Drawing the grid using rectangles
- Rendering X and O symbols based on board values
- Displaying the current player indicator
- Drawing a line across winning cells

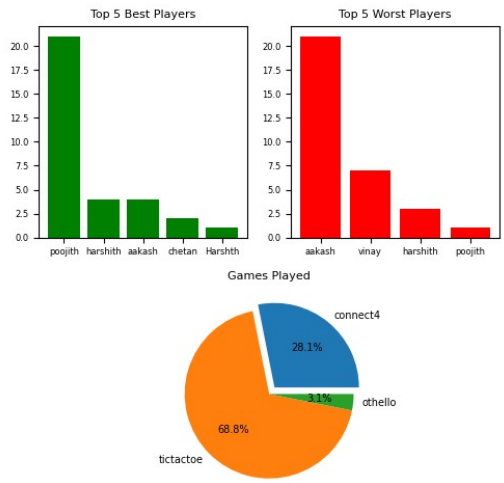

Figure 7: Statistics

3.1.4 Move Handling

The `position_in_table()` function:

- Detects mouse click events
- Converts screen coordinates into board indices
- Returns the selected row and column

The `make_move()` function:

- Places the player's symbol if the cell is empty
- Calls win-check logic after each move
- Checks for draw condition if the board is full
- Switches player turn if the game continues

3.1.5 Win Detection Logic

The game checks for a win using sliding window technique from NumPy:

- **Horizontal Check:** A window of size 5 slides across each row. If all values match the current player, a win is declared.
- **Vertical Check:** A window of size 5 slides across each column to detect vertical alignment.
- **Diagonal Check (Top-Left to Bottom-Right):** A 5×5 sub-matrix is extracted and its main diagonal is checked.
- **Diagonal Check (Top-Right to Bottom-Left):** The sub-matrix is flipped and its diagonal is checked.

If any condition is satisfied:

- Winning cell positions are stored
- A line is drawn across those cells
- Game state is updated to game over

3.1.6 Game Loop

The `run()` function controls the main loop:

- Continuously updates the screen
- Handles user input and game state
- Stops when a player wins or the game is a draw

3.1.7 Result Storage

After the game ends:

- The result is saved into a CSV file
- Player names, date, and game type are recorded

3.1.8 Conclusion

The implementation efficiently combines event handling, graphical rendering, and optimized win detection using NumPy. The use of sliding window technique ensures fast checking of winning conditions even on a larger grid.

3.2 Othello

3.2.1 Overview

The Othello game is implemented on an 8×8 grid where two players alternately place discs and flip opponent pieces. A move is valid only if it captures at least one opponent piece in any direction.

3.2.2 Board Representation

The board is represented using a NumPy 2D array:

- 0 represents an empty cell
- 1 represents Player 1 (Black disc)
- 2 represents Player 2 (White disc)

3.2.3 Initial Setup

The function `settingup_initial_pieces()` initializes the standard Othello starting configuration with four discs placed in the center of the board.

3.2.4 Direction-Based Move Logic

The game uses an 8-direction vector list: [(-1,-1), (-1,0), (-1,1), (0,-1), (0,1), (1,-1), (1,0), (1,1)] Each move is evaluated in all directions to determine valid flips.

3.2.5 Valid Move and Flip Detection

The function `_flips_for_move()` performs the core logic:

- Checks if the selected cell is empty
- Traverses in all 8 directions
- Collects opponent pieces in a straight line
- Confirms a valid move only if a player's piece encloses opponent pieces
- Returns all pieces that must be flipped

3.2.6 Valid Moves Generation

The function `get_valid_moves()` scans the entire board and collects all positions that result in at least one valid flip for the current player.

3.2.7 Move Execution

The `make_move()` function:

- Accepts player input (mouse click)
- Validates move using flip detection logic
- Places the player's disc on the board
- Flips all opponent discs in valid directions
- Switches player turn after a successful move

3.2.8 Win Condition Logic

The function `check_win()` determines the end of the game:

- The game ends when no valid moves remain for both players OR the board is full
- Player scores are calculated using NumPy sum:
  - Player 1 score = number of cells equal to 1
  - Player 2 score = number of cells equal to 2
- The player with the higher score is declared the winner

3.2.9 User Interaction Handling

The function `position_in_table()`:

- Captures mouse click events
- Maps screen coordinates to board positions
- Handles menu button navigation

3.2.10 Game Loop

The `run()` function controls the main gameplay loop:

- Continuously updates the board display
- Processes player moves
- Checks game termination conditions

3.2.11 Result Storage

At the end of the game:

- Winner and loser details are stored in a CSV file
- Date and game type are recorded for history tracking

3.2.12 Conclusion

The Othello implementation uses direction-based search logic for move validation and flipping mechanics. The design efficiently handles game state updates, move validation, and scoring using NumPy operations and structured event handling.

3.3 Connect4

3.3.1 Overview

The Connect 4 game is implemented using a 7×7 grid where two players take turns dropping discs into columns. The objective is to connect four consecutive discs horizontally, vertically, or diagonally.

3.3.2 Board Representation

The board is represented using a 2D NumPy array:

- 0 represents an empty cell
- 1 represents Player 1
- 2 represents Player 2

3.3.3 Rendering the Board

The `draw_board()` function:

- Draws the game background
- Renders circular slots for each cell
- Displays player discs based on board values
- Shows the current player indicator
- Displays a menu button

3.3.4 Column Selection Logic

The `check_coloum()` function:

- Detects mouse click events
- Checks if the click is inside any circular slot
- Returns the column index corresponding to the clicked position

3.3.5 Move Execution

The `make_move()` function:

- Receives the selected column
- Iterates from bottom row upwards
- Places the disc in the lowest available position
- Checks for win condition after placing the disc
- Checks for draw if the board is full
- Switches player turn if the game continues

3.3.6 Win Detection Logic

Winning conditions are checked using NumPy sliding window technique:

- **Horizontal Check:** A window of size 4 moves across rows to find consecutive matches.
- **Vertical Check:** A window of size 4 moves across columns.
- **Diagonal Check (Top-Left to Bottom-Right):** A 4×4 submatrix is extracted and its main diagonal is verified.
- **Diagonal Check (Top-Right to Bottom-Left):** The submatrix is flipped and its diagonal is checked.

If any condition is satisfied:

- The game is marked as over
- The current player is declared the winner

3.3.7 Game Loop

The `run()` function:

- Continuously renders the game screen
- Handles player input
- Executes moves until game over condition

3.3.8 Result Handling

After the game ends:

- The result is stored in a CSV file
- Player names, date, and game type are recorded

3.3.9 Conclusion

The implementation uses efficient array operations and event-driven programming. The use of sliding window technique ensures fast and optimized detection of winning patterns in the grid.

4 Hurdles, issues faced during the project and overcoming them

4.1 Hurdles

- Learning and using python took a lot of time, understanding syntax, all functions.
- learning and understanding pygame without being taught in class
- learning and using numpy while doing project in a very short time, trying to use numpy slicing instead nested loops
- Writing the games function like check_win, make_move etc
- making sure all rules of games are followed
- Checking the logical errors in code

4.2 Overcoming the hurdles

- We learnt python by referring the slides provided and python tutorial (W3schools[7][8] and python[4])
- We learnt pygame from online tutorials.[3]
- seeing the slides of numpy provided and numpy references, later using them and learning their syntax and usage.[6][5][2]
- From the game info & rules, writing the functions and later adding necessary helper functions required
- By playing game, checking whether the game rule is obeyed or not, ensuring game flow is same as what predicted from rules.
- Constantly running the game, looking for errors, bugs in code by checking where rules may have been violated/disobeyed, if any rule is violated, game flow is not as expected, code is thoroughly checked for issues.

5 Possible project improvements if extra time is permitted/alloted

if we extra time is provided/permitted we would have

- Make display more creative and colourful.
- We would have added another game in addition to the present three games.
- Menus design would have been made more clean and add more necessary options
- Make the game code more cleaner, short and compact.

6 References

References

[1] Matplotlib Developers. Matplotlib tutorials, 2026.

[2] NumPy Developers. Numpy reference documentation, 2026.

[3] Pygame Community. Pygame documentation, 2026.

[4] Python Software Foundation. The python tutorial, 2026.

[5] W3Schools. Numpy reference, 2026.

[6] W3Schools. Numpy tutorial, 2026.

[7] W3Schools. Python reference, 2026.

[8] W3Schools. Python tutorial, 2026.